

PSD Assignment 1 - 16-bit Multicycle CISC Processor Implementation

Aravind Sunder S

Sr. No. 27233

M.Tech EPD, DESE

Indian Institute of Science Bangalore-560012

ABSTRACT

This report documents the implementation and validation of a 16-bit multicycle CISC processor, synthesized for the Basys3 FPGA platform (Xilinx Artix-7 XC7A35T). Building on the base architecture - an 8-register register file, 16-bit shared-bus datapath, 5-bit addressing, and a unified 64×16 -bit distributed-memory unit - this report presents an unsigned integer division algorithm using the repeated-subtraction method, operating on two 16-bit operands stored at memory addresses 0x20 and 0x21. The algorithm computes both quotient and remainder, exposed on the Basys3 LEDs and seven-segment display via a dedicated top-level wrapper that taps internal register-file signals.

I PROCESSOR ARCHITECTURE

1.1 Overview

The processor implements a 16-bit shared-bus architecture. All major functional units - the register file, ALU, program counter, instruction register, and memory - communicate over a common 16-bit data bus, arbitrated by a multiplexer (mux module) so that only one source drives the bus in any given cycle. This prevents bus contention and simplifies the datapath control logic to a small set of enable/load signals generated by the FSM controller.

1.2 Top Level Modules

The design consists of the following hierarchy:

- **basys3 top** - Board-level wrapper. Instantiates the CPU core, taps register R1 (remainder) and R3 (quotient) directly via hierarchical reference into the register bank, drives the 16 LEDs with R1, and multiplexes R3 onto a 4-digit seven-segment display.
- **top** - CPU core. Instantiates the controller, register decoder, register bank, memory, program counter, ALU, pre-instruction decoder, and output data mux.
- **controller** - FSM-based control unit generating all datapath control signals.
- **reg decode** - Combinational decoder mapping 3-bit rd/rs1/rs2 fields to individual register load/enable lines.
- **register bank / register** - Eight 16-bit registers (R1–R8) with per-register load and read-enable control.
- **memory** - 64×16 -bit distributed RAM (Xilinx dist mem gen 0 core), COE-initialized.
- **PC** - 16-bit program counter with load and increment control.
- **ALU** - Performs ADD, SUB, INC, DEC, SHIFTL, and RRC (rotate-right-through-carry), with registered CARRY and combinational ZERO flags.
- **pre ins decoder (prefetch reg + ins register)** - Two-stage instruction pipeline: a prefetch register (TMP) that speculatively fetches the next sequential instruction while the current one decodes, and the instruction register (IR) that holds the instruction currently executing.
- **mux** - Final bus-source multiplexer selecting between memory output, PC value, register-file output, and ALU output.

1.3 Datapath Block Diagram

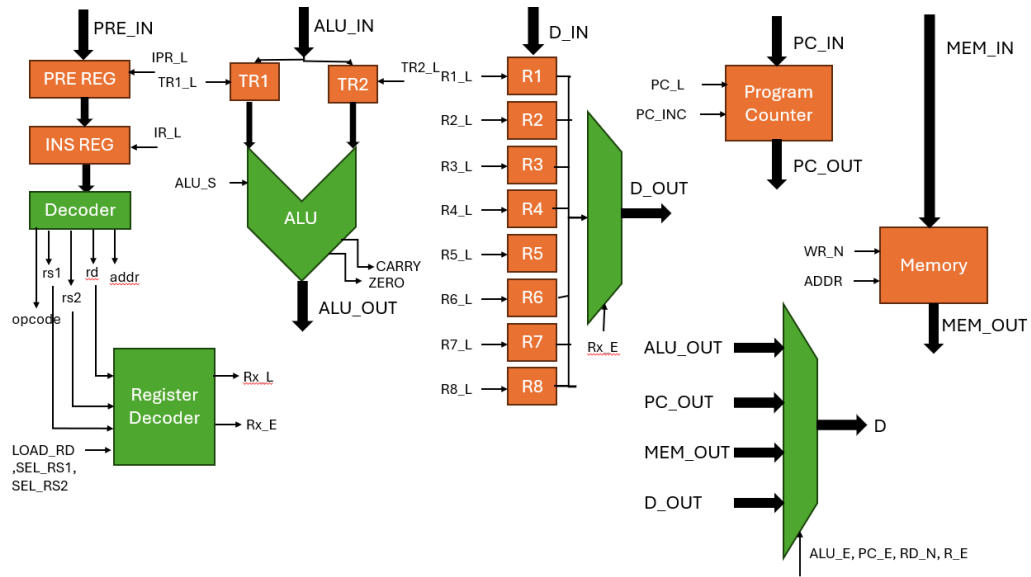


Figure 1: Datapath Architecture

II CONTROLLER FSM

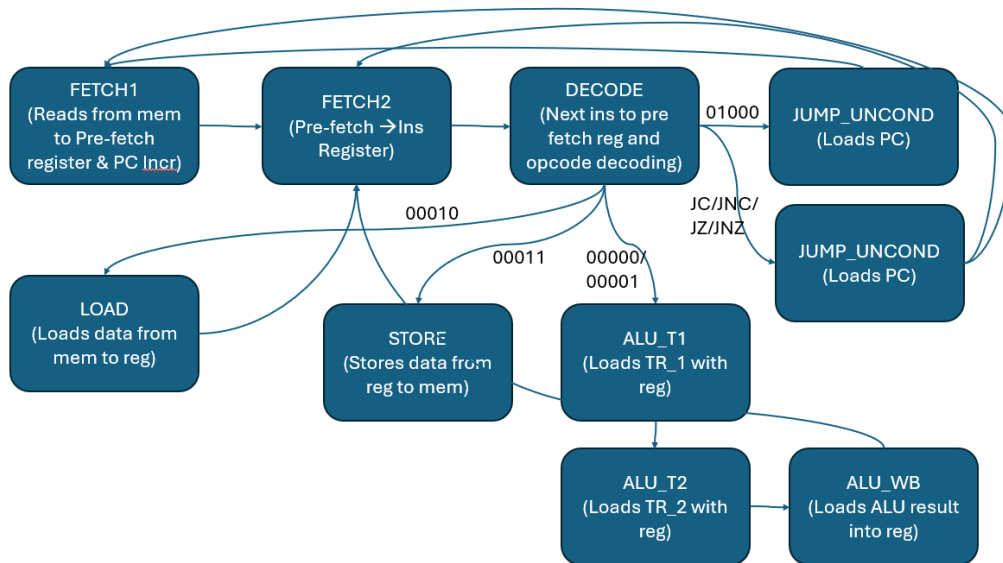


Figure 2: Controller FSM

III INSTRUCTION SET ARCHITECTURE

Table 1: Instruction Opcode Encoding (as implemented in controller)

Opcode (Binary)	Instruction	Mnemonic
00000	Add	ADD
00001	Subtract	SUB
00010	Load	LD
00011	Store	ST
00100	Increment	INC
00101	Decrement	DEC
00110	Shift left	SHIFTL
00111	Rotate right through carry	RRC
01000	Jump unconditional	JMP
01001	Jump if zero	JZ
01010	Jump if not zero	JNZ
01011	Jump if carry	JC
01100	Jump if no carry	JNC

Table 2: Instruction Word Fields (16-bit)

Field	Bits	Width	Notes
opcode	[15:11]	5	Operation code
rd	[10:8]	3	Destination register (R1–R8)
rs2	[7:5]	3	Second source register (ADD/SUB)
rs1	[4:2]	3	First source register (ADD/SUB)
address	[4:0]	5	Memory/jump target (LD/ST/JMP/Jcond)

IV DIVISION ALGORITHM: REPEATED SUBTRACTION

4.1 Problem Statement

Given two unsigned 16-bit operands stored in data memory at addresses 0x20 (dividend) and 0x21 (divisor), compute the integer quotient and remainder using the repeated-subtraction method: the divisor is subtracted from a running remainder as many times as possible without the result going negative; the subtraction count is the quotient, and the final non-negative remainder is the division remainder.

4.2 Algorithm

1. Load the dividend into a working register (running remainder), the divisor into a second register, and initialize a quotient counter to zero.
2. Loop: subtract the divisor from the running remainder.
3. If the result of the subtraction underflows (i.e. the running remainder would go below zero, detected via the CARRY/borrow flag from the ALU's subtract operation), the loop terminates — the last valid remainder is retained and the subtraction that caused underflow is not committed.
4. Otherwise, commit the subtraction result as the new running remainder, increment the quotient counter, and repeat.
5. On termination, store the quotient and remainder to memory (and/or hold them in dedicated registers for external display).

4.3 Hardware Exposure of Results

Per the `basys3_top` wrapper, register R1 is tapped as the remainder output (displayed directly on the 16 board LEDs) and register R3 is tapped as the quotient output (multiplexed across the four seven-segment digits at an approximately 190 Hz per-digit refresh rate, using a common-anode active-low hex decoder).

V PROGRAM IMPLEMENTATION

5.1 Memory Initialization (COE File)

The processor's unified instruction/data memory is initialized via the following COE vector. The program performs the repeated-subtraction division of the two 16-bit operands held at addresses 0x20 and 0x21.

```
memory_initialization_radix = 16;
memory_initialization_vector =
1014,
1115,
1218,
1418,
0B20,
580B,
6008,
400B,
008C,
2208,
4004,
1856,
1817,
400D,
400E,
400F,
4010,
4011,
4012,
4013,
0019,
0004,
0000,
0000,
0000,
0001,
0000,
0000,
0000,
```

5.2 Assembly-Level Description

```
LD R1, [0x20] ; load dividend / initial operand
LD R2, [0x21] ; load divisor
LD R3, [0x24] ; initialize quotient to zero
LD R5, [0x24] ; load zero onto R5 – Zero register
```

```
LOOP:
SUB R4, R1, R2 ; dividend – divisor
JC DONE ; borrow (underflow) → quotient limit reached, exit
JNC CONTINUE ; no borrow → subtraction valid, keep looping
```

```
CONTINUE:
ADD R1, R4, R5 ; load new remainder to R1
INC R3 ; store intermediate remainder
JMP LOOP
```

```
DONE:
    ST  R3, [0x22]      ; store final quotient
    ST  R1, [0x23]      ; store final remainder
HERE:
    JMP HERE           ; halt sequence
```

VI FPGA IMPLEMENTATION RESULTS

6.1 Timing Report

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 1.179 ns		Worst Hold Slack (WHS): 0.175 ns		Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 549		Total Number of Endpoints: 549		Total Number of Endpoints: 240	
All user specified timing constraints are met.					

Figure 3: Timing Report

Maximum Clock Frequency = $1/(10-1.179) = 113.6\text{Mhz}$

6.2 Power Report

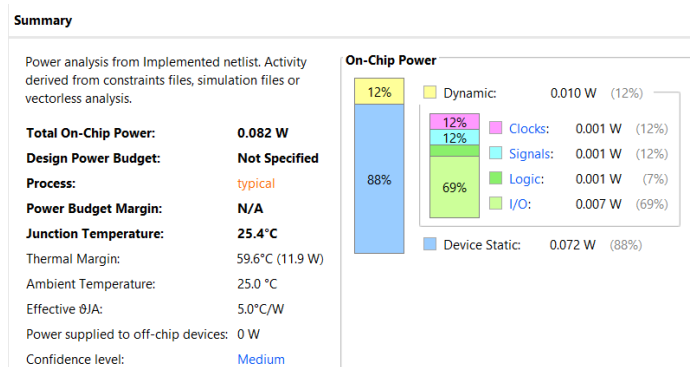


Figure 4: Power Report

6.3 Post Implementation Simulation

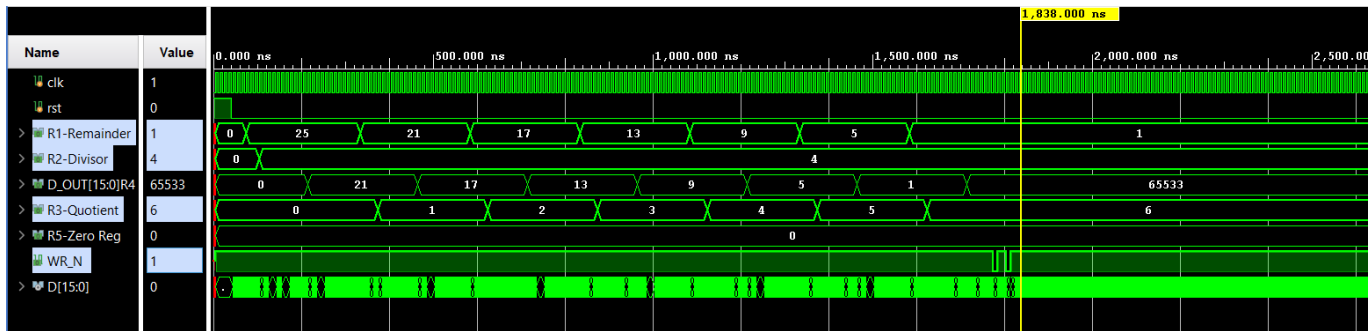


Figure 5: Post Implementation Simulation Results

6.4 Resource Utilization

Utilization			
		Post-Synthesis	Post-Implementation
Graph Table			
Resource	Utilization	Available	Utilization...
LUT	265	20800	1.27
LUTRAM	16	9600	0.17
FF	223	41600	0.54
IO	29	106	27.36
BUFG	1	32	3.13

Figure 6: Resource Utilization

VII CONCLUSION

This report extends the validated 16-bit multicycle CISC processor with a repeated-subtraction unsigned division workload operating on two 16-bit operands stored in data memory. The algorithm leverages the ALU's subtract operation and registered borrow (CARRY) flag, combined with the controller's conditional-jump states (JC/JNC), to iteratively determine the quotient and remainder without requiring dedicated division hardware. The quotient and remainder are exposed on the Basys3 board via the seven-segment display and LEDs respectively, using hierarchical signal taps into the register file. This confirms the processor's control-flow and ALU borrow-detection logic function correctly for iterative, branch-driven arithmetic algorithms.